

Experiment # 09

File Structure related System Calls in Linux

Objective

In this lab we will learn about system call techniques in LINUX.

Software Tools

- Ubuntu 16.04
- GCC Compiler
- Online Compiler https://www.onlinegdb.com/online_c_compiler

Theory

System Call:

A system call is just what its name implies, a request for the operating system to do something on behalf of the users' program.

Process related system calls:

- `fork()`
- `wait()`
- `exec()`
- `exit()`

Basically there are total 5 types of I/O or File Structure system calls:

- `create()`
- `open()`
- `close()`
- `read()`
- `write()`

creat()

1. **Create:** Used to create a new empty file.

Syntax in C language:

```
int creat(char *filename, mode_t mode)
```

Parameter:

- **filename :** name of the file which you want to create
- **mode :** indicates permissions of new file.

Returns:

- return first unused file descriptor (generally 3 when first creat use in process because 0, 1, 2 fd are reserved)
- return -1 when error

How it work in OS

- Create new empty file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry

- Return file descriptor used, -1 upon failure

Example

```
#include<stdio.h>
#include<fcntl.h>
#include<errno.h>
int main()
{
    int fd=creat("foo1.txt",O_CREAT);
    if (fd ==-1)
    {
        // print program detail "Success or failure"
        perror("Error Creating file");
    }
    else
    {
        printf("Successful");
    }
    return 0;
}
```

open()

open: Used to Open the file for reading, writing or both.

Syntax in C language

```
#include<sys/types.h>
#include<sys/stat.h>
#include <fcntl.h>
int open (const char* Path, int flags [, int mode ]);
```

Parameters:

- **Path :** path to file which you want to use
 - use absolute path begin with “/”, when you are not work in same directory of file.
 - Use relative path which is only file name with extension, when you are work in same directory of file.
- **flags :** How you like to use
 - **O_RDONLY:** read only, **O_WRONLY:** write only, **O_RDWR:** read and write, **O_CREAT:** create file if it doesn’t exist, **O_EXCL:** prevent creation if it already exists

How it works in OS

- Find existing file on disk
- Create file table entry
- Set first unused file descriptor to point to file table entry
- Return file descriptor used, -1 upon failure

Example Syntax:

- `open("hello.txt", O_RDWR | O_CREAT)` // hello.txt is opened/created and is ready for reading and writing.
- `open("hello.txt", O_WRONLY | O_CREAT);` // hello.txt is opened/created and is ready for writing only.
- `open("hello.txt", O_RDONLY | O_CREAT);` // hello.txt is opened/created and is ready for reading only.
- If successful open returns non negative integer else return -1.

Example

```
#include<stdio.h>
#include<fcntl.h> //used for file related system calls
#include<errno.h> //tells type of errors
int main()
{
    // if file does not have in directory
    // then file foo.txt is created.
    int fd = open("foo.txt", O_RDONLY | O_CREAT);

    if (fd == -1)
    {
        // print program detail "Success or failure"
        perror("Error Creating file");
    }
    else
    {
        printf("Successful");
    }
    return 0;
}
```

close()

close: Tells the operating system you are done with a file descriptor and Close the file which pointed by fd.

Syntax in C language

```
#include <fcntl.h>
int close(int fd);
```

Parameter

- **fd** :file descriptor

Return

- **0** on success.
- **-1** on error.

How it works in the OS

- Destroy file table entry referenced by element fd of file descriptor table
 - As long as no other process is pointing to it!
- Set element fd of file descriptor table to **NULL**

Example

```

#include<stdio.h>
#include<errno.h>
#include <fcntl.h>
int main()
{
    int fd1 = open("foo.txt", O_RDONLY);
    if (fd1 < 0)
    {
        perror("error opening file\n");
    }
    else
        printf("opened the file\n");

    // Using close system Call
    if (close(fd1) < 0)
    {
        perror("error closing file\n");
    }
    else
        printf("closed the file");
    return 0;
}

```

read()

read: From the file indicated by the file descriptor fd, the read() function reads cnt bytes of input into the memory area indicated by buf. A successful read() updates the access time for the file.

Syntax in C language

```
size_t read (int fd, void* buf, size_t cnt);
```

Parameters

- **fd:** file descriptor
- **buf:** buffer to read data from
- **cnt:** length of buffer

Returns: How many bytes were actually read

- return Number of bytes read on success
- return 0 on reaching end of file
- return -1 on error
- return -1 on signal interrupt

Important points

- **buf** needs to point to a valid memory location with length not smaller than the specified size because of overflow.
- **fd** should be a valid file descriptor returned from open() to perform read operation because if fd is NULL then read should generate error.
- **cnt** is the requested number of bytes read, while the return value is the actual number of bytes read. Also, sometimes read system call should read less bytes than cnt.

Example

```
#include<stdio.h>
#include<fcntl.h>
#include <unistd.h>
int main()
{
    static char buffer[80];
    int fd = open("foo.txt",O_RDONLY);
    if (fd != -1)
    {
        printf("myfile opened for read access\n");
        read(fd, buffer, 100);
        printf("read from my file is as follows:\n%s ", buffer);
        close (fd);
    }
    return 0;
}
```

Example: Reading data from two files

```
#include<stdio.h>
#include<fcntl.h>
#include <unistd.h>
int main()
{
    static char c[100];
    static char c1[100];
    int fd1 = open("foo1.txt", O_RDONLY);
    int fd2 = open("foo.txt", O_RDONLY);
    read(fd1, c, 100);
    read(fd2, c1, 100);
    printf("c = %s \n", c);
    printf("c = %s \n", c1);
    return 0;
}
```

Example: Reading single character from file

```
#include<stdio.h>
#include<fcntl.h>
#include <unistd.h>
#include <stdlib.h> //for exit()
int main()
{
    char c;
    int fd1 = open("foo.txt", O_RDONLY, 0);
    read(fd1, &c, 1);
    printf("c = %c\n", c);
}
```

```
    exit(0);
}
```

write()

write: Writes cnt bytes from buf to the file or socket associated with fd. cnt should not be greater than INT_MAX (defined in the limits.h header file). If cnt is zero, write() simply returns 0 without attempting any other action.

Syntax in C language

```
#include <fcntl.h>
```

```
size_t write (int fd, void* buf, size_t cnt);
```

Parameters

- **fd:** file descriptor
- **buf:** buffer to write data to
- **cnt:** length of buffer

Returns: How many bytes were actually written

- return Number of bytes written on success
- return 0 on reaching end of file
- return -1 on error
- return -1 on signal interrupt

Important points

- The file needs to be opened for write operations
- **buf** needs to be at least as long as specified by cnt because if buf size less than the cnt then buf will lead to the overflow condition.
- **cnt** is the requested number of bytes to write, while the return value is the actual number of bytes written. This happens when **fd** have a less number of bytes to write than cnt.
- If write() is interrupted by a signal, the effect is one of the following:
 - If write() has not written any data yet, it returns -1 and sets errno to EINTR.
 - If write() has successfully written some data, it returns the number of bytes it wrote before it was interrupted.

Example: Writing data in file

```
#include<stdio.h>
#include<fcntl.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    static char buffer[80];
    static char message[] = "Hello, world";
    int fd = open("foo.txt",O_RDWR | O_CREAT);
    if (fd != -1)
    {
        printf("myfile opened for writing\n");
        write(fd, message, sizeof(message));
        close (fd);
    }
```

```

}
    return 0;
}

```

Example: Writing data in file

```

#include<stdio.h>
#include<fcntl.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    static char buffer[80];
    char message[10];
    printf("\nwrite in your file:");
    scanf("%10[^\t]s", message);
    printf("%s",message);
    int fd = open("foo.txt",O_RDWR | O_CREAT);
    if (fd != -1)
    {
        printf("myfile opened for writing\n");
        write(fd, message, sizeof(message));
        close (fd);
    }
    return 0;
}

```

Implementation of open(), read(), write() and close() System Calls

```

#include<stdio.h>
#include<fcntl.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    char buffer[80];
    static char message[] = "Hello,world";
    int fd = open("myfile",O_RDWR |O_CREAT);
    if (fd != -1)
    {
        printf("myfile opened for read/writeaccess\n");
        write(fd, message, sizeof(message));
        lseek(fd, 0, 0); /* go back to the beginning of the file */
        read(fd, buffer, sizeof(message));
        printf("%s was written to myfile \n", buffer);
        close (fd);
    }
    return 0;
}

```

Lab Tasks

1(a). **Simulate** suitable command/commands in gedit editor to open a file which should have permission of writing only. Now write five city names in the file. And add screenshot of .txt file in your lab report.

1(b). **Simulate** suitable command/commands in gedit editor to open a file which should have permission of reading only. Now read above written city names from the file. And show contents in the output.

2. **Simulate** suitable command/commands in gedit editor to write your introduction i.e. name, father's name, registration number and class in the file and after that read those characters from file and save it into a variable. Display the contents of variable i.e. data read from the file.

Conclusion:

Lab Instructor
Fasih Ahmad